

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Institution of Manipal University

MINOR SPECIALISATION FINAL REPORT

TITLE OF PROJECT

Secure and Efficient Decentralised Key Management Framework
for Peer to Peer Distributed Systems

TEAM MEMBERS

Dhruv Kaushik Thacker - 230905566 - School of Computer Engineering
Errol D Almeida - 230905239 - School of Computer Engineering

MINOR SPECIALISATION NAME AND THE DEPARTMENT OFFERING

CYBER SECURITY - COMPUTER SCIENCE DEPARTMENT

NAME OF GUIDE

Dr. RadhaKrishna Bhat

DATE OF REPORT

5 November 2026

ABSTRACT

This project implements secure peer communication with locally owned cryptographic identities, signed key lifecycle transactions and a native ledger at each peer. The current Python terminal application uses an optional bootstrap directory to discover listening addresses. Peers then establish direct TCP connections for messages, key updates and chain synchronization. Established communication survives bootstrap shutdown. The implementation includes persistent identity files, encrypted direct messages, group channels, revocation, reinstatement, rotation and signed message-hash receipts.

Direct messages combine P-256 elliptic-curve key agreement, HKDF-SHA-256, authenticated encryption, and ECDSA signatures. Revocation updates are pushed to connected contacts identified from channel membership and direct interaction. Other peers retrieve identity state when required and cache verification results for a default of 300 seconds. This design reduces unnecessary immediate notifications in sparse interaction graphs while retaining explicit refresh and timeout handling. Signed DATA transactions record message hashes and routing metadata without placing direct-message plaintext on the ledger.

The updated implementation passed 32 local automated tests across direct mesh, revocation regression and receipt suites, plus the scoped benchmark and crypto/DPKI checks. Tests confirm operation after bootstrap shutdown. The team previously reported successful multi-device LAN operation of the earlier version; this updated version requires a fresh LAN run.

The implementation distributes key ownership, validation and application transport across peers. The bootstrap is an optional address directory and cannot forward application traffic. New peers may instead contact an existing peer and obtain addresses through peer exchange. In-memory ledger storage, public-name-derived channel keys, static key agreement and limited consensus remain prototype limitations.

Keywords: peer-to-peer communication, decentralised key management, DPKI, ECDSA, revocation cache, native blockchain, message receipts.

CONTENTS

1 INTRODUCTION 4
2 PROBLEM STATEMENT 5
3 LITERATURE SURVEY 6
4 OBJECTIVES 8
5 METHODOLOGY AND SYSTEM DESIGN 9
6 IMPLEMENTATION DETAILS 13
7 RESULTS AND RESULT ANALYSIS 17
8 CONCLUSIONS AND FUTURE SCOPE 20
9 REFERENCES 22
APPENDIX A DEMONSTRATION PROCEDURE 23
APPENDIX B REPRODUCIBILITY AND SOURCE BASIS 24

Abbreviations

Term	Meaning
AEAD	Authenticated Encryption with Associated Data
DID	Decentralized Identifier
DPKI	Decentralized Public Key Infrastructure
ECDH	Elliptic Curve Diffie-Hellman key agreement
ECDSA	Elliptic Curve Digital Signature Algorithm
HKDF	HMAC-based Extract-and-Expand Key Derivation Function
LAN	Local Area Network
TTL	Time to Live

1 INTRODUCTION

1.1 Project context

Secure peer communication requires more than encrypting a message. A sender must obtain the correct public key, decide whether that key remains valid, and respond when the owner changes or revokes it. A receiver must authenticate the sender and reject altered or unauthorised data. These requirements become harder when participants join and leave dynamically and no certificate authority manages every identity.

The project investigates these problems through a working communication prototype. Every peer controls a P-256 key pair and a DID-shaped identity document. A local ledger records signed registration, revocation, reinstatement, rotation, and message-receipt transactions. Peers use that ledger alongside an in-memory registry, trust scores, and a revocation cache to make communication decisions [9].

1.2 Current implementation

The current execution path starts with peer.py and peer_cli.py. PeerNode runs a direct TCP listener and manages peer connections. relay.py supplies listening addresses only. Identity announcements, direct messages, channels, broadcasts, receipts and chain synchronization travel over direct connections. Each endpoint performs cryptographic checks and retains its own state.

The repository also retains JavaScript, libp2p, Solidity, and Hardhat files from earlier development. They are not required by the current Python CLI. The native ledger is implemented in blockchain.py and does not execute EVM smart contracts. This distinction is necessary when explaining deployment, demonstrating the project, or interpreting the earlier documents.

1.3 Development outcome

The earlier demonstration used relay-routed communication. The current implementation retains the existing key management and ledger functionality while replacing application forwarding with direct TCP connections, connection recovery and peer exchange. Bootstrap availability no longer controls peer membership or key-state refresh.

The contribution is an integrated and testable teaching prototype, rather than a new cryptographic primitive. The project combines established algorithms with an interaction-based notification policy and an inspectable local ledger. Successful local tests and a team-reported LAN demonstration support its functional objectives. Production availability, recovery, private-group membership, and strong distributed consensus remain outside the demonstrated scope.

2 PROBLEM STATEMENT

2.1 Core problem

Design and implement a peer communication framework that lets participants generate and retain their own keys, distribute verifiable identity records, authenticate and protect messages, and manage key revocation and replacement without a central key-issuing authority. The framework must make key-state changes visible to relevant peers and avoid unnecessary network-wide notifications during ordinary operation.

A public key announcement alone is insufficient. A peer may possess an outdated key, continue to trust an identity after revocation, or remain quarantined after legitimate reinstatement. Network delay can also cause an implementation to check its old state before a requested update arrives. These failures affect both confidentiality decisions at the sender and acceptance decisions at the receiver.

2.2 Engineering challenges

- Identity continuity: retain the same peer identity across restarts and associate key changes with signed lifecycle records.
- Freshness and efficiency: notify current contacts promptly while allowing unrelated peers to retrieve state on demand.
- Concurrent operation: process incoming chat verification asynchronously so direct readers can receive chain responses without blocking.
- Message authenticity: verify signatures on direct messages, channel messages, broadcasts, and ledger transactions.
- Auditability: expose key history and message-hash records without storing confidential message plaintext on the ledger.
- Practical deployment: run the same source on multiple LAN devices with a small, reproducible Python setup.

2.3 Scope and assumptions

The system assumes a reachable bootstrap or a known peer address for initial discovery, and reachable peer listening ports on the LAN. Established peers communicate without the bootstrap. Signed transactions prove control of keys, not real-world identity. The direct connection handshake associates a socket with a peer ID but does not implement cryptographic proof of possession. Payload signature checks remain unchanged.

A completed result for this scope is a runnable CLI, working encrypted and signed communication, observable revocation and reinstatement, key rotation, receipt anchoring, and repeatable tests. The report evaluates those capabilities separately from stronger claims such as perfect forward secrecy, immutable global finality, or instantaneous revocation knowledge at every node.

3 LITERATURE SURVEY

3.1 Key lifecycle management

NIST SP 800-57 Part 1 Revision 5 describes key management as a collection of functions and protection requirements for cryptographic keying material [1]. This perspective is directly relevant: generating a strong key does not resolve storage, replacement, revocation, or recovery. The project therefore treats lifecycle operations as explicit transactions and CLI commands. Its current plaintext identity files and same-key reinstatement also show where the prototype falls short of a complete operational key-management policy.

3.2 Digital signature selection

Serengil and Ozpinar present LightDSA, a configurable digital-signature library and a performance study covering RSA, DSA, ECDSA, and EdDSA under different key, hash, and curve configurations [2]. The study motivates separating cryptographic choices from communication logic and measuring configurations in context. The present implementation follows that architectural idea through provider interfaces, while its asymmetric provider is specifically P-256. It does not implement every algorithm evaluated by LightDSA or establish a new comparative performance result.

3.3 Decentralised identity and communication

DIAP illustrates that identity, proof, discovery and transport are separate design concerns [3]. This project has a narrower implementation: local DID-shaped documents, signed key transactions, direct TCP communication and optional bootstrap discovery. IPFS, IPNS, zero-knowledge proofs, GossipSub and Iroh are not implemented.

W3C DID Core defines an identifier and document model through which a controller can express verification methods and related information [4]. The project uses the did:peer prefix and includes a verification method referencing its public key. This supports the project's internal identity representation, but no DID-method interoperability or formal conformance test is included. The report therefore does not repeat the earlier blanket claim of W3C compliance.

3.4 Relation to the earlier synopsis

The supplied synopsis and presentation established the initial motivation and surveyed a wider set of topics, including homomorphic encryption and privacy-preserving learning. Those topics are background rather than implemented components of this messaging system. The final survey emphasises literature and specifications that explain the current design, while preserving the original focus on peer-owned identity, authentication, and key lifecycle management.

3 LITERATURE SURVEY CONTINUED

3.5 Key derivation and authenticated encryption

RFC 5869 specifies HKDF as an extract-and-expand construction for deriving cryptographic key material [5]. In this project, an ECDH shared secret is passed to HKDF-SHA-256 to produce a 32-byte symmetric key. This separates the shared-secret computation from the key supplied to the authenticated cipher. The code uses long-lived peer keys and a fixed context string, so this construction alone does not provide an ephemeral session handshake or forward secrecy.

RFC 8439 describes the ChaCha20 and Poly1305 construction, while the cryptography library exposes authenticated-encryption interfaces for both ChaCha20-Poly1305 and AES-GCM [6, 7]. The application selects one of these providers and generates a fresh 12-byte nonce for each encryption operation. An AEAD authentication tag detects ciphertext alteration when used with the correct key and nonce. Application identity and lifecycle decisions remain separate checks around the cipher operation.

3.6 Hash-linked ledgers and distributed agreement

Nakamoto's design combines hash-linked transactions, proof of work, network propagation, and a chain-selection rule under explicit assumptions about competing computation [8]. The project adopts the educational ideas of signed transactions, linked blocks, and a longer-valid-chain preference. Its default proof-of-work difficulty is one leading hexadecimal zero, blocks are produced locally, and there is no production validator or incentive system. The resulting ledger is a prototype audit structure rather than a security equivalent of a deployed public blockchain.

Reference area	Contribution to this project	Remaining gap
Key management [1]	Lifecycle operations and explicit state	Protected storage and independent recovery
Signatures [2]	Modular crypto provider interfaces	Broader algorithm evaluation
Identity [3, 4]	Peer-owned identifier and verification method	DID-method interoperability and proofs
HKDF and AEAD [5-7]	Key derivation and protected message payloads	Ephemeral sessions and context binding
Ledger design [8]	Signed history and chain validation	Durable consensus and finality

The design opportunity addressed by the project is integration: combine messaging, local identity state, lifecycle events, and a measurable notification policy in one understandable application. Its experiments evaluate functional correctness and a small scoped-delivery scenario. They do not establish superiority over the cited systems or substitute for formal protocol analysis.

4 OBJECTIVES

The overall objective is to demonstrate peer-owned key management with authenticated communication and observable lifecycle controls. The following objectives translate the original proposal into capabilities of the current implementation.

Objective	Implemented mechanism	Verification basis
Generate and retain identity	P-256 key pair, local identity JSON, DID-shaped document	Code inspection and identity/key tests
Discover and communicate	Direct TCP mesh, optional bootstrap, DM, channels and broadcast	Network scripts and team LAN demonstration
Authenticate and protect messages	ECDSA checks, ECDH/HKDF and AEAD	Crypto and signature regression tests
Manage key lifecycle	Signed register, revoke, reinstate and rotate transactions	DPKI and revocation suites
Reduce immediate notification fan-out	Connected interaction contacts and lazy verification cache	Four-peer experiment with 14 checks
Provide an inspectable audit record	Native ledger, key history and message-hash receipts	Receipt tests and CLI inspection
Support repeatable evaluation	CLI commands, metrics and six automated test scripts	Recorded execution of the current source

4.1 Acceptance criteria

A valid message must reach the intended application path and pass the relevant identity, signature, and encryption checks. A known revoked identity must be blocked. A valid reinstatement must update state so communication can resume. Repeating a transaction must not replay its lifecycle event or add a duplicate block. A failed connected-peer refresh must not create a fresh verified-active cache entry.

For the efficiency objective, the experiment must identify the actual contact graph, count immediate scoped deliveries, and compare them with the number of other peers. For auditability, the ledger must store and retrieve a signed receipt for each tested message type. These acceptance criteria measure the implementation directly and avoid treating a displayed status or a successful socket write as proof of end-to-end delivery.

5 METHODOLOGY AND SYSTEM DESIGN

5.1 System architecture

Every peer runs the same Python application, maintains a separate identity file and listens for direct TCP connections. The optional bootstrap directory lists peer IDs and listening addresses. It supplies discovery notifications without becoming an application-message path [9].

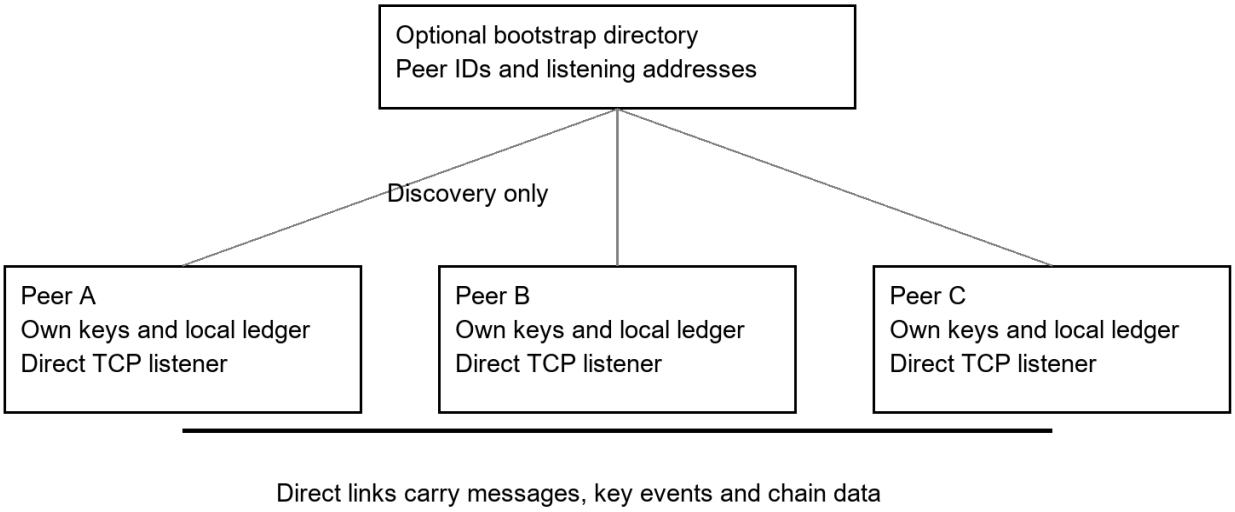


Figure 1 Direct TCP mesh with optional bootstrap discovery

The CLI invokes asynchronous PeerNode methods. The node checks identity state, encrypts or signs application data, creates any associated receipt and writes a newline-delimited JSON envelope directly to the recipient. Broadcasts are sent once to each connected peer. Receivers validate protocol payloads before updating local state or displaying messages.

The bootstrap receives peer identifiers and listening addresses. Application messages, receipts and chain data are exchanged directly. The directory explicitly rejects application forwarding. Bootstrap disconnection leaves live peer sockets, memberships, pending chain requests and revocation caches intact. Discovery remains dependent on an address source for a newcomer, while established communication does not depend on one central server.

5.2 State ownership

Persistent state consists of the peer ID, private key, public key, and DID document saved in an identity JSON file. The local ledger, key history, trust scores, channel memberships, revocation cache, and message history are held in memory. A restarted peer can learn state from connected peers, but stopping all peers loses the complete in-memory history. This distinction informs both the restart procedure and future persistence work.

5.3 Identity lifecycle and ledger model

At startup, a peer loads an existing identity file or generates a new key pair. Its default peer ID is a generated label with a libp2p-like prefix; it is not derived from the public key. The node builds an internal DID document and submits a signed registration transaction to its local ledger. Identity announcements carry the registration transaction, allowing a receiver to validate the key-binding record rather than accepting an unsigned key string.

Transaction	Authorisation and effect
REGISTER_KEY	Creates or updates a peer key record according to the ledger validation rules; a new registration must prove possession of its supplied key.
REVOKE_KEY	Signed by the current registered key; marks the identity revoked and records a reason.
UNREVOKE_KEY	Signed by the registered key of a revoked identity; restores active status.
ROTATE_KEY	Signed by the current active key; replaces the public key and retains key history.
DATA	Requires an active registered signer; message receipts use the MESSAGE_RECEIPT action.

A transaction records its type, sender peer ID, payload, nonce, timestamp, and signature. A block records an index, timestamp, transactions, previous hash, nonce, and hash. The transaction hash excludes its signature and is then signed using ECDSA. The ledger uses SHA-256 for its own hashing independently of the configurable message-hash provider.

5.4 Validation and chain synchronisation

Submitting a transaction validates its signature and permitted state transition, then mines a local block immediately. Duplicate transaction hashes are recognised without replaying their effects. A received full chain is checked against the deterministic genesis block, block indices, previous hashes, computed hashes, proof-of-work prefix, duplicate transactions, and replayed transaction validity.

A longer valid chain can replace the local chain. A requested response from a peer may also contain newer signed transactions for that peer on a shorter fork; the synchronisation path can import applicable records and reconcile local state. This addresses a freshness case in the tests, but does not provide general fork merging or global finality. Chain length, local timestamps, and peer-provided history remain limitations of the prototype agreement model.

5.5 Protected message workflow

Direct messaging first verifies the recipient's current key state, then computes an ECDH shared secret from the sender's private key and recipient's public key. HKDF-SHA-256 derives the AEAD key. A fresh nonce is used to encrypt the text, and the encrypted fields plus timestamp are signed. The receiver verifies sender state and signature before decrypting and displaying the message.

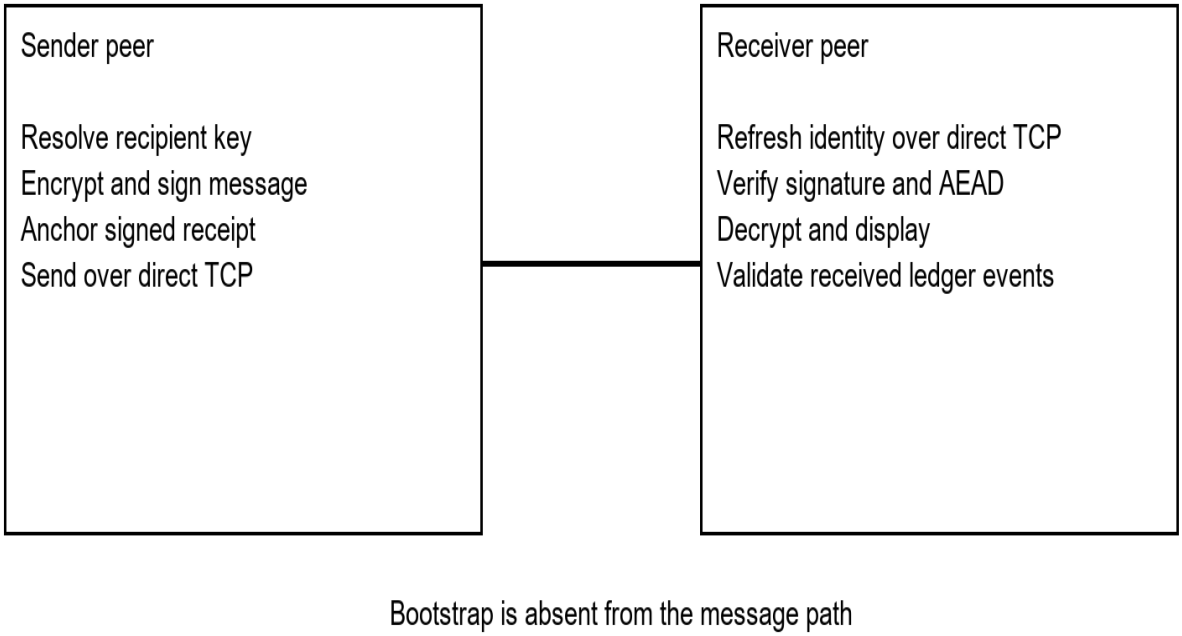


Figure 2 Direct-message processing with a sender-created ledger receipt

A channel message signs the channel name, ciphertext, nonce, tag and timestamp. The sender distributes it through direct connections; receivers check channel membership and signatures. Public broadcasts sign plaintext and are intentionally unencrypted. All three message types support automatic receipt creation.

The design uses independent checks: encryption protects a payload under its derived key, signatures authenticate signed fields, and the ledger supplies identity status. None of these checks proves availability or delivery. A receipt may already exist when a later socket write fails, and no receiver-signed acknowledgement is implemented.

5.6 Scoped revocation and lazy verification

Each peer derives an interaction set from members of its joined channels, retained direct-message history, and recorded direct presence. Only connected members of that set receive a scoped lifecycle push. All peers initially join #general, so that shared channel must be left before demonstrating separated groups. A successful direct message can also add a contact relationship.

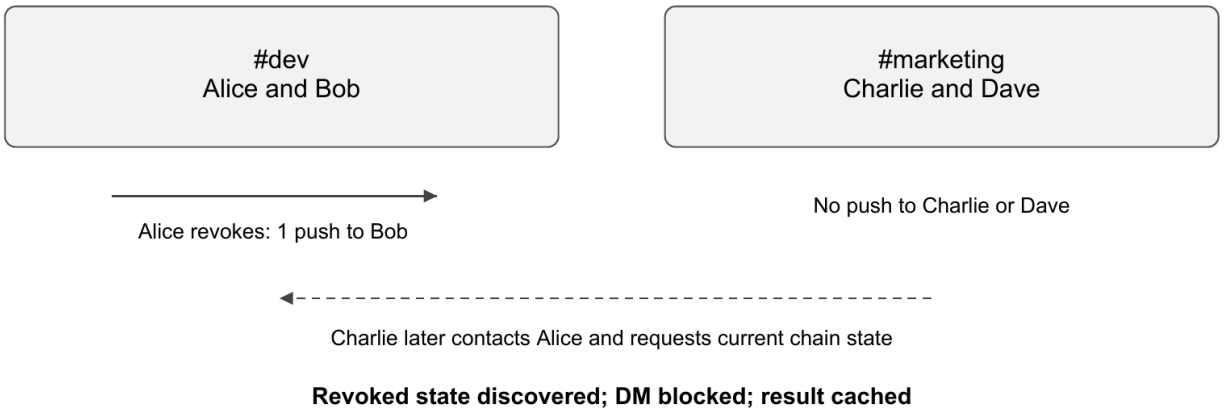


Figure 3 Separate groups reduce immediate notification fan-out while retaining on-demand checks

When verification is requested, a valid unexpired cache entry can be reused. Otherwise, a connected target is asked for its chain. Requests carry identifiers, concurrent requests for the same target share a pending response, and the default timeout is two seconds. If the refresh fails, the stale entry is removed and verification returns SYNC_UNAVAILABLE rather than a fresh active result.

Accepted lifecycle events reconcile the key registry, revoked set, trust score and cache. Unchanged synchronization does not extend cache lifetime. A cached revoked state can be refreshed to discover reinstatement. Running peers perform fresh chain requests over direct connections; an unavailable refresh returns an unverified status. Chat verification runs in background tasks so readers can process its response.

5.7 Freshness trade-off

The default positive-cache TTL is 300 seconds. Contacts can learn revocation through a push, while an unrelated peer may retain an earlier active entry until expiry or explicit refresh. The policy therefore reduces immediate notification traffic at the cost of bounded cache staleness under ordinary operation. It does not guarantee that a malicious or isolated peer will disclose the latest revocation history. The refresh command provides an explicit user-controlled check.

6 IMPLEMENTATION DETAILS

6.1 Module organisation

Module	Responsibility
peer.py	Small entry point that launches the terminal application.
peer_cli.py	Interactive menu, commands, status, trust matrix, metrics, ledger and receipt views.
peer_node.py	Identity loading, messaging, membership, lifecycle operations, cache, synchronisation and reconnection.
blockchain.py	Transactions, local blocks, signature/state validation, key histories and receipt lookup.
crypto_modular.py	Hash, asymmetric and AEAD provider interfaces and the CryptoSuite coordinator.
crypto.py	Common key, DID, signature, hash and encryption helpers, including ledger-facing primitives.
relay.py	Discovery-only TCP directory returning peer listening addresses.
test_*.py	Automated unit, regression, component and network verification.

6.2 Asynchronous transport

The optional bootstrap listens on TCP 4001. Peers register their IDs and direct listening addresses. Peers then establish bidirectional TCP connections with a HELLO and acknowledgement exchange. Concurrent connection attempts select one connection deterministically. Cleanup of an obsolete socket cannot remove its replacement. Received envelopes must match the connected peer ID.

Discovery notifications supply connection candidates. Direct connection establishment triggers identity, chain, channel-state and peer-address exchange. Direct disconnection updates membership and invalidates the affected peer cache. Heartbeats and retry backoff recover direct connections without the bootstrap. Bootstrap retries run independently and never clear established peer state.

These transport mechanisms support independent LAN operation. They do not add TLS, a cryptographically authenticated handshake or a new consensus algorithm. The blockchain and cryptographic providers remain unchanged.

6.3 Cryptographic providers and trust state

Component	Current implementation
Asymmetric primitive	NIST P-256 for ECDSA signatures and ECDH shared-secret computation.
Signature digest	SHA-256 inside the P-256 signing provider.
Message hash providers	SHA-256, BLAKE2b with a 256-bit digest, and SHA3-256.
Key derivation	HKDF-SHA-256; 32-byte output; empty salt; fixed application context string.
AEAD providers	AES-256-GCM and ChaCha20-Poly1305 with 12-byte random nonces and 16-byte tags.
Default suite	SHA-256, P-256 and AES-256-GCM.
Alternative suite	BLAKE2b-256, P-256 and ChaCha20-Poly1305.

The provider interfaces make algorithm choices explicit and allow the CLI to swap the message suite. Corresponding peers must select compatible settings; suite negotiation and protocol-level downgrade protection are not implemented. Switching the message hash does not change ECDSA's SHA-256 digest, HKDF-SHA-256, or the ledger's SHA-256 hashing.

6.4 Local trust scoring

A newly learned active identity starts with a local trust score of 50. A successfully verified and decrypted direct message increases the score by five, capped at 100. A bad direct-message or channel signature reduces it by 25, floored at zero. Accepted revocation sets the score to zero; reinstatement can restore the baseline of 50. These values are local indicators rather than a distributed voting or reputation protocol.

The actual security gates are identity validity, revocation state, signature checks, and decryption success. A high displayed score cannot replace those checks. The current code does not implement the periodic uptime bonus or peer-voting mechanism discussed in the earlier presentation.

6.5 Storage and channel behaviour

Identity JSON files contain unencrypted private key material and must remain private to their owners. Message history retains the latest 100 entries and the event log retains the latest 200 entries. Neither is a durable audit database. Channel keys are deterministically derived from public channel names, so channel encryption demonstrates the messaging path without enforcing private membership. Real confidential groups require a separate authenticated key-distribution design.

6.6 Message receipts and terminal audit

The updated messaging methods accept anchor_on_chain=True by default. Before transmitting a message, the sender computes a message hash and commits a signed DATA transaction with action MESSAGE_RECEIPT. Its payload includes the message type, target or recipient, and message hash. The receipt is propagated to current contacts and its metadata is attached to the message envelope [9].

Message type	Input used for the receipt hash
Direct message	Ciphertext, IV, authentication tag and timestamp, joined in the same order as the signature payload.
Channel message	Channel name followed by ciphertext, IV, tag and timestamp.
Public broadcast	Plaintext message followed by its ECDSA signature.

The receiver stores the attached metadata and a locally recomputed message hash with its message-history record. The receipts or audit command lists local ledger receipts and supports a message-hash lookup. A plaintext query works only when the local retained message history supplies the associated stored hash. It is not a general plaintext-to-ciphertext search and should not be described as independent proof that a recipient read a message.

Receipt verification in the current helper is a lookup for a matching hash in the local ledger. It does not itself rerun whole-chain validation or automatically authenticate every metadata field attached to an incoming envelope. Signed receipt transactions, normal transaction validation, and chain checks provide the surrounding audit context. Strengthening the binding between the received envelope, the signed receipt, and the displayed verification result is future work.

6.7 Changes from the earlier demonstration

Earlier material	Current Python implementation
JavaScript/libp2p execution	Python CLI with direct asyncio TCP peer connections.
Key announcements and signed chat	Signed lifecycle transactions, encrypted DMs, channels and public broadcasts.
Persistence and revocation listed as next steps	Identity files, revoke, reinstate, rotate, scoped updates and cache reconciliation.
Basic peer join and leave visibility	Direct connection recovery, membership exchange and independent bootstrap retry.
No message audit feature	Automatic message-hash receipts, ledger explorer and audit commands.

6.8 Debian and LAN deployment

The application requires the same current Python source on every device, Python 3.11 or newer for the supplied regression tests, and cryptography>=42.0.0. A virtual environment keeps the project dependency separate from Debian's system Python. The current CLI does not require Node.js, npm, Hardhat, an Ethereum account, or a public blockchain connection.

Device A runs the discovery bootstrap and a peer; Devices B and C run peers. Discovery uses Device A TCP 4001. Every peer exposes TCP 5001 in this example and must permit inbound LAN connections. Each peer advertises its own LAN IP. Client isolation and host firewalls can prevent direct connections even when discovery succeeds.

Environment setup on each device

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip \
  ca-certificates netcat-openbsd
cd ~/minor-lan/p2p-simple
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Relay on Device A

```
RELAY_HOST=0.0.0.0 RELAY_PORT=4001 python relay.py
```

Peers in separate terminals

```
# Device A, second terminal
python peer.py --port 8501 --listen-port 5001 --advertise-host 192.168.1.100 --relay 192.168.1.100:4001
# Device B
python peer.py --port 8502 --listen-port 5001 --advertise-host 192.168.1.101 --relay 192.168.1.100:4001
# Device C
python peer.py --port 8503 --listen-port 5001 --advertise-host 192.168.1.102 --relay 192.168.1.100:4001
```

Activate the environment in each terminal. The --port option selects an identity filename; --listen-port selects the actual TCP listener. Permit inbound TCP 5001 on each peer and TCP 4001 on the bootstrap host. Use distinct identities and do not transfer private identity files. For multiple peers on one computer, assign distinct listening ports. The companion Execution Guide gives complete Debian commands.

The project team reports that the updated application worked across devices on a LAN. The automated measurements in Section 7 were obtained separately on localhost. Appendix A provides a short demonstration sequence covering the current lifecycle and receipt commands.

7 RESULTS AND RESULT ANALYSIS

7.1 Evaluation method

The updated transport was verified locally on Windows using the bundled Python 3.12 runtime. Thirty-two automated tests passed across the direct mesh, revocation regression and message receipt suites. The 14-check scoped-revocation benchmark, modular cryptography suite and native DPKI suite also passed. CLI startup without a bootstrap was checked. These results concern local TCP tests, not a new multi-device Debian run.

Test script	Observed result	Scope
test_peer_revocation_ regressions.py	20 tests passed	Cache, reinstatement, signatures, invalid chains, delayed replies, timeout and reconnect cases.
test_message_receipts.py	2 tests passed	Ledger receipts and received DM, channel and broadcast messages with receipt metadata.
test_efficient_revocation.py	14 checks passed	Four-peer scoped notifications, lazy detection and 1,000 cached lookups.
test_modular_crypto.py	Completed successfully	Hash and AEAD providers, suites, messaging and revoked-sender blocking.
test_dpki.py	Completed successfully	Local ledger, key lifecycle, encryption and forged-signature rejection.
test_network_e2e.py	Completed successfully	10 direct-mesh tests: bootstrap failure, messaging, key updates, receipts and peer exchange.

The direct-mesh suite contains 10 tests, the revocation regression suite 20 tests, and the message receipt suite two tests. All 32 passed. Additional benchmark and cryptographic scripts completed successfully. These checks exercise specific scenarios and do not constitute exhaustive validation.

7.2 Separate LAN evidence

The team confirmed successful operation on a LAN using multiple devices. This supports the functional deployment objective beyond a single-machine loopback demonstration. No device inventory, packet capture, load profile, latency distribution, or independently recorded delivery ratio accompanied that confirmation. Accordingly, the LAN result is reported as a successful team-run functional test, without attaching unmeasured throughput or reliability figures.

7.3 Scoped revocation experiment

Four peers discovered each other and formed direct connections. Alice and Bob shared #dev; Charlie and Dave shared #marketing. All peers left #general. No cross-group DM was sent before revocation. Alice therefore had only Bob as an active application contact, despite the full transport mesh.

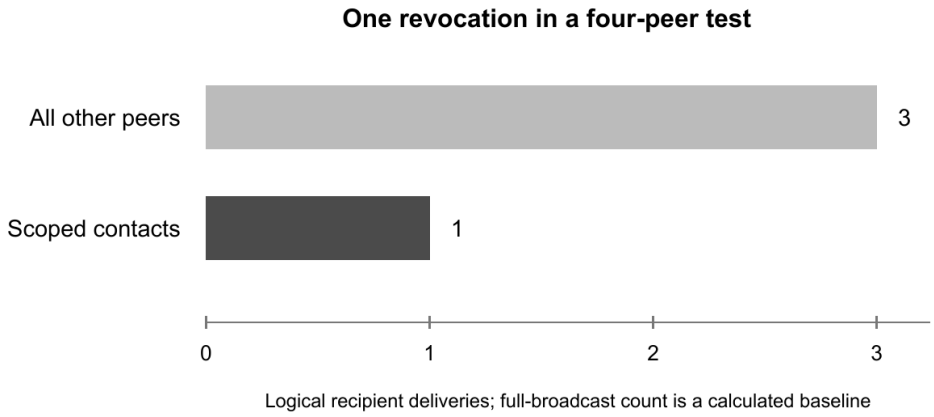


Figure 4 Immediate notification recipient count in the four-peer experiment

Observation	Recorded value
Alice's scoped revocation deliveries	1
Calculated all-other-peers baseline	3
Bob's received revocation notifications	1
Charlie and Dave notifications	0 each
Charlie's first DM attempt after revocation	Blocked after a cache miss and chain refresh
Repeated cache lookups	1,000 hits; 0.98 ms total; 0.98 microseconds average

The benchmark retained one scoped delivery instead of three calculated broadcast deliveries, a reduction of 66.7 percent. This measures logical recipient deliveries, not bandwidth. Discovery, heartbeats, chain refreshes and receipt propagation are outside that comparison.

The retained timing of 0.98 milliseconds for 1,000 cache hits comes from the earlier local benchmark snapshot. It is an in-process cache timing, not a measurement of the new direct transport or LAN latency. The updated benchmark also passed all 14 checks; timing values should be remeasured on the deployment devices.

7.4 Functional findings and interpretation

Revocation and reinstatement

The regression suite verifies that accepted revocation updates override cached active state and that reinstatement reconciles both explicit events and replaced-chain state. A three-node sequence demonstrates a previously unrelated peer discovering revocation through a lazy query, blocking communication, discovering later reinstatement, and resuming messaging. This directly addresses the stale-quarantine behaviour observed during earlier development.

Asynchronous verification and failure handling

Delayed chain replies use matching request identifiers, and concurrent requests share one pending result. A timeout cannot create a fresh verified entry. Tests confirm that a pending refresh survives bootstrap shutdown, scoped revocations reach contacts, strangers refresh directly and reinstatement and key rotation continue to work.

Authentication and ledger integrity checks

Tests reject unsigned channel and public messages, an unsigned DATA transaction, and a forged revocation included in a candidate chain. Invalid revocation cannot quarantine a peer, and duplicate transactions do not replay events. These checks support the claimed validation behaviour for the tested cases. They do not provide proof against every malformed input, replay attack, malicious relay, or adversarial fork.

Message-receipt functionality

The receipt tests register an identity, anchor receipts for DM, channel, and broadcast traffic, retrieve their fields, and check a missing-hash lookup. The network test sends all three message types between two PeerNode instances and verifies the receiver’s message history and attached receipt metadata. Thus, receipt creation and ordinary delivery paths work together in the exercised scenario. Receiver-signed delivery confirmation and automatic strict receipt-to-envelope validation are not covered capabilities.

Result boundaries

The experiments use small peer sets and local sockets. The suite includes a measured cache loop and a logical notification count, but not a large-network stress test. Moreover, the scoped-send counter also includes other scoped transactions, including receipts and later lifecycle updates. Meaningful manual comparisons should take before-and-after deltas around one operation. Shared channels or prior DM history can change the contact set and therefore the number of recipients.

Overall, the results support functional correctness of the implemented flows and demonstrate reduced immediate fan-out in one sparse graph. They leave open the cost of full-chain refresh, concurrent writes and forks, sustained message receipt growth, network partitions, and operation under malicious participants.

8 CONCLUSIONS AND FUTURE SCOPE

8.1 Conclusions

The project delivers a working Python framework for peer-owned identities, authenticated communication, and explicit key lifecycle management. The current implementation supports persistent key files, encrypted direct messaging, channels, signed public broadcasts, revocation, reinstatement, rotation, a local native ledger, and message-hash receipts. Its terminal interface exposes state and results without requiring an EVM or a graphical application.

The scoped-notification policy and lazy verification cache connect security state to actual interactions. The four-peer experiment delivered one immediate revocation update where a broadcast would target three peers, while an unrelated peer discovered the revoked state on demand. Regression tests confirm recovery from stale cache state, waiting for delayed replies, rejection of selected invalid inputs, and resumed communication after reinstatement.

Local verification supports the updated transport: messaging, receipts, key lifecycle operations, direct reconnection and joining through an existing peer work after bootstrap shutdown. The team previously reported a successful multi-device LAN demonstration of the earlier version. The new version must be retested on those devices using the companion Execution Guide. Local test results do not establish production-grade consensus or complete resistance to adversarial peers.

8.2 Current limitations

- Discovery and identity admission: a newcomer needs a bootstrap address or a known peer address. Direct handshakes do not prove key possession. Peer-ID spoofing and adversarial address announcements remain security limitations.
- Durability and agreement: identity files persist, but the complete ledger does not. Local mining and longer-chain replacement can diverge or discard history; difficulty one offers no robust finality.
- Confidentiality boundaries: long-lived keys are reused for ECDH and signing. There is no ephemeral session protocol or forward secrecy, and public channel names determine group keys.
- Freshness and recovery: cached active state can remain stale, peer-provided history is not an independent freshness oracle, and the same key can reinstate a revoked identity.
- Audit semantics: a sender-created receipt records a commitment, not delivery. Receipt metadata exposes communication relationships, and message replay protection is incomplete.

8.3 Future development

The next stage should preserve the inspectable CLI and modular code while strengthening the trust and persistence model. Priorities follow the gaps exposed by the implementation rather than introducing unrelated features.

- Durable state and recovery: persist validated ledger state and key history, define safe restart reconciliation, encrypt or otherwise protect private keys at rest, and support independent recovery authorisation after compromise.
- Authenticated transport: add proof of possession during direct handshakes, authenticated address advertisements and appropriate transport protection. Retain bootstrap-independent operation.
- Session and group security: separate signing and key-agreement keys, introduce ephemeral authenticated sessions, bind sender, recipient, protocol and suite identifiers into authenticated context, and distribute private channel keys only to authorised members.
- Freshness and agreement: specify an adversarial model, improve fork handling and conflict rules, and evaluate how a verifier obtains current revocation status independently of the peer being checked.
- Stronger receipts: validate received metadata against the signed transaction and local message hash, add message identifiers and replay suppression, and use receiver-signed acknowledgements when delivery evidence is required.
- Evaluation at scale: collect repeated latency and throughput measurements across physical devices, record actual bytes and chain sizes, introduce partitions and delayed packets, and test increasing numbers of peers and concurrent lifecycle changes.
- Interoperability and maintainability: validate DID-method representations, define versioned protocol schemas, add explicit suite negotiation, and expand tests for malformed envelopes and receipt mismatches.

A useful evaluation sequence is to address durable state and authenticated admission first, then strengthen session and group security, and finally measure the resulting system under load and failures. This keeps the security model understandable and ensures that performance improvements are evaluated against clearly stated guarantees.

The completed prototype provides a practical base for this work: it already separates cryptography, ledger logic, messaging, and CLI operations, and includes repeatable tests for the key-state transitions that most directly affect communication.

9 REFERENCES

[1] E. Barker, Recommendation for Key Management: Part 1 - General, NIST SP 800-57 Part 1 Rev. 5, May 2020. https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final

[2] S. Serengil and A. Ozpinar, LightDSA: A Python-Based Hybrid Digital Signature Library and Performance Analysis of RSA, DSA, ECDSA and EdDSA in Variable Configurations, Elliptic Curve Forms and Curves, arXiv:2505.23773, 2025. https://arxiv.org/abs/2505.23773

[3] Y. Liu, W. Xing, Y. Zhou, G. Chang, C. Lin, and M. Han, DIAP: A Decentralized Agent Identity Protocol with Zero-Knowledge Proofs and a Hybrid P2P Stack, arXiv:2511.11619, 2025. https://arxiv.org/abs/2511.11619

[4] W3C, Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations, W3C Recommendation, 19 July 2022. https://www.w3.org/TR/2022/REC-did-core-20220719/

[5] H. Krawczyk and P. Eronen, HMAC-based Extract-and-Expand Key Derivation Function (HKDF), RFC 5869, May 2010. https://www.rfc-editor.org/rfc/rfc5869.html

[6] Y. Nir and A. Langley, ChaCha20 and Poly1305 for IETF Protocols, RFC 8439, June 2018. https://www.rfc-editor.org/rfc/rfc8439.html

[7] Python Cryptographic Authority, Authenticated Encryption, cryptography documentation, version 50.0.2 consulted for this report. https://cryptography.io/en/stable/hazmat/primitives/aead/

[8] S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. https://bitcoin.org/bitcoin.pdf

[9] Project source, current local Python implementation reviewed on 5 November 2026: peer_node.py, peer_cli.py, blockchain.py, crypto_modular.py, crypto.py, relay.py, peer.py and requirements.txt. Local changes were included; the report is not limited to a previously published repository revision.

[10] Project verification records, November 2026: direct mesh acceptance suite, revocation regression suite, message receipt tests, scoped benchmark and crypto/DPKI checks. Earlier local verification records remain in tmp/report_verification; mesh verification logs are stored in tmp/mesh_*.log. Local evidence is separate from the team-reported earlier LAN demonstration.

APPENDIX A DEMONSTRATION PROCEDURE

A1 Establish the test network

Start the relay and three peers using Section 6.8. On every peer, run peers and status, record the actual peer IDs, and set crypto sha256 aes-gcm. Replace ALICE_ID, BOB_ID and CHARLIE_ID below with those IDs. The commands in this appendix run inside the peer CLI unless explicitly identified as shell commands.

A2 Demonstrate scoped revocation before cross-group messages

Alice and Bob	Charlie
join #dev leave #general	join #marketing leave #general

On Bob, run revoke 1. Alice should receive the lifecycle update and mark Bob revoked. Charlie should receive no scoped revocation push while it remains outside Bob's contact set. On Charlie, run dm BOB_ID test-after-revoke. The lazy verification should discover revocation and block the DM. Inspect metrics, trust and lookup BOB_ID.

On Bob, run unrevoke. Retry Charlie's DM. The previously known revoked state triggers a refresh so that valid reinstatement can be discovered and the message can be accepted. Check the receiving terminal as well as the sender status. A successful DM changes the interaction graph, so repeat this experiment with fresh processes and the same separate-channel setup when comparing packet counts.

A3 Demonstrate communication and receipts

```
dm BOB_ID hello-from-Alice
#dev signed-channel-message
broadcast signed-public-message
receipts
chain
blocks
```

Confirm the DM at Bob, the channel message at its joined members, and the broadcast at other active peers. Use receipts or audit to inspect the locally available ledger entries. A listed receipt is evidence of a recorded signed hash; it is not a recipient acknowledgement. Answer n to leave the receipt lookup prompt, or enter y followed by a recorded message hash to inspect a match.

A4 Rotation and reconnect

On Bob, run rotate and status. Refresh Bob from Alice and send another DM. Stop the bootstrap while peers remain running; repeat messaging, revocation, reinstatement, rotation and refresh. Existing memberships must remain intact. Restart a peer with its original identity file and --no-relay --peer HOST:PORT to join through a running peer.

APPENDIX B REPRODUCIBILITY AND SOURCE BASIS

B1 Automated verification commands

Run the following in a shell from p2p-simple with the virtual environment active. The scripts use isolated local test relays and check their own scenarios; they are separate from the live demonstration relay on port 4001.

```
python -B test_peer_revocation_regressions.py
python -B test_message_receipts.py
python -B test_efficient_revocation.py
python -B test_modular_crypto.py
python -B test_dpki.py
python -B test_network_e2e.py
```

For this report, the scripts were executed from a temporary copy of the current Python files. This avoided changing live participant identity files. The saved verification manifest records Python and cryptography versions, source SHA-256 values, process exit codes, and elapsed script times. Those elapsed times include setup and deliberate waits and are not reported as message latency.

B2 Source snapshot identifiers

peer_node.py

1037f15b5680621bbfa16322e0d4aec3cc9e1e5346deac5c4da2ee7ca847d6c6

blockchain.py

5168aeed015146aee1f4ed258ad83f596122301b3e47aac50e317b5104ee5d1a

peer_cli.py

8e900efe2feda98a0e52178372c7b653712815949a0eebdb000c97e7eb3dd76c

B3 Use of the supplied project documents

MINOR SPECIALISATION SYNOPSIS supplies the cover layout, typography, project title, specialisation, guide and retained team details. The Outlook evaluation mail supplies the order of the eight main technical sections. Minor.pdf and the earlier execution-procedure report provide the development baseline and LAN demonstration context. Their JavaScript commands, proposed mechanisms, old performance claims and old screenshots are not presented as evidence of the current Python system.

The final report uses the latest reviewed local implementation as the technical authority. Literature statements are supported by the references in Section 9. Automated results come from the recorded execution, while the successful physical LAN result is attributed to the project team. No unrecorded device count, throughput, latency distribution, or faculty feedback is inferred.